

BASE DE DATOS

SISTEMA DE VENTAS

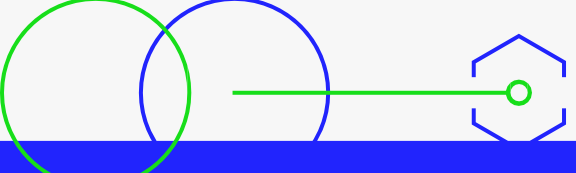
HERNANDEZ ESPINOZA

GABRIEL EMANUEL

DESARROLLADOR

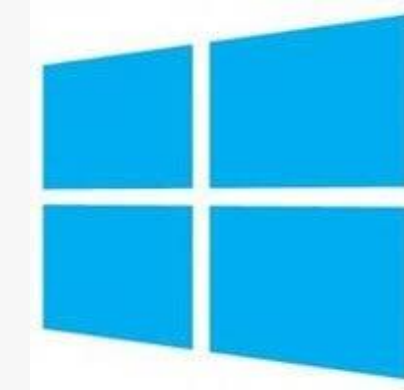


**HERNANDEZ ESPINOZA
GABRIEL EMANUEL**



CARACTERÍSTICAS DEL PROYECTO

- Aplicación de Escritorio.
- Para pequeñas y medianas empresas.
- Generación de reportes en PDF
- Generación de archivos en EXCEL



LIBRERIAS

AbsoluteLayout.jar

Permite diseñar interfaces gráficas de usuario (pantallas) colocando los elementos (botones, cuadros de texto, etc.) exactamente donde uno quiera, como si estuvieras acomodando cosas en una hoja.

swingx-all-1.6.4.jar

Agrega componentes visuales adicionales a los que ya trae Java, como calendarios, listas mejoradas, o paneles con más funciones. Mejora la apariencia de las ventanas y formularios.

commons-collections4-4.1.jar

Ayuda a manejar mejor listas, conjuntos y otros tipos de datos en el programa. Es como una caja de herramientas para trabajar con información organizada.

poi-3.16.jar

poi-ooxml-3.16.jar

poi-ooxml-schemas-3.16.jar

xmlbeans-2.6.0.jar

Permiten trabajar con archivos de Microsoft Excel (como leer o escribir hojas de cálculo). Es útil si tu programa necesita exportar o leer datos en formato .xls.

itextpdf-5.5.1.jar

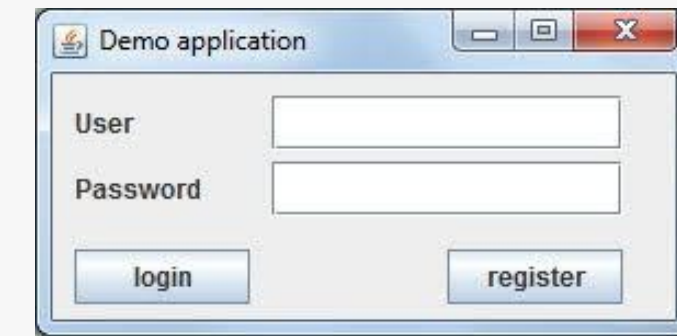
Permite crear y modificar archivos PDF desde tu programa. Por ejemplo, puedes generar un reporte en PDF con los datos que tiene el sistema.

sqlite-jdbc-3.49.1.0.jar

Es lo que conecta tu programa con una base de datos SQLite. Así puedes guardar y consultar información (como usuarios, ventas, etc.) desde una base de datos.

JDK 24 (Default)

Es el "motor" que permite escribir y ejecutar programas en Java. Sin esto, nada funcionaría. Es como el lenguaje que entiende la computadora para correr tu sistema.



CONEXION ENTRE EL SISTEMA Y SQLITE

```
package Modelo;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

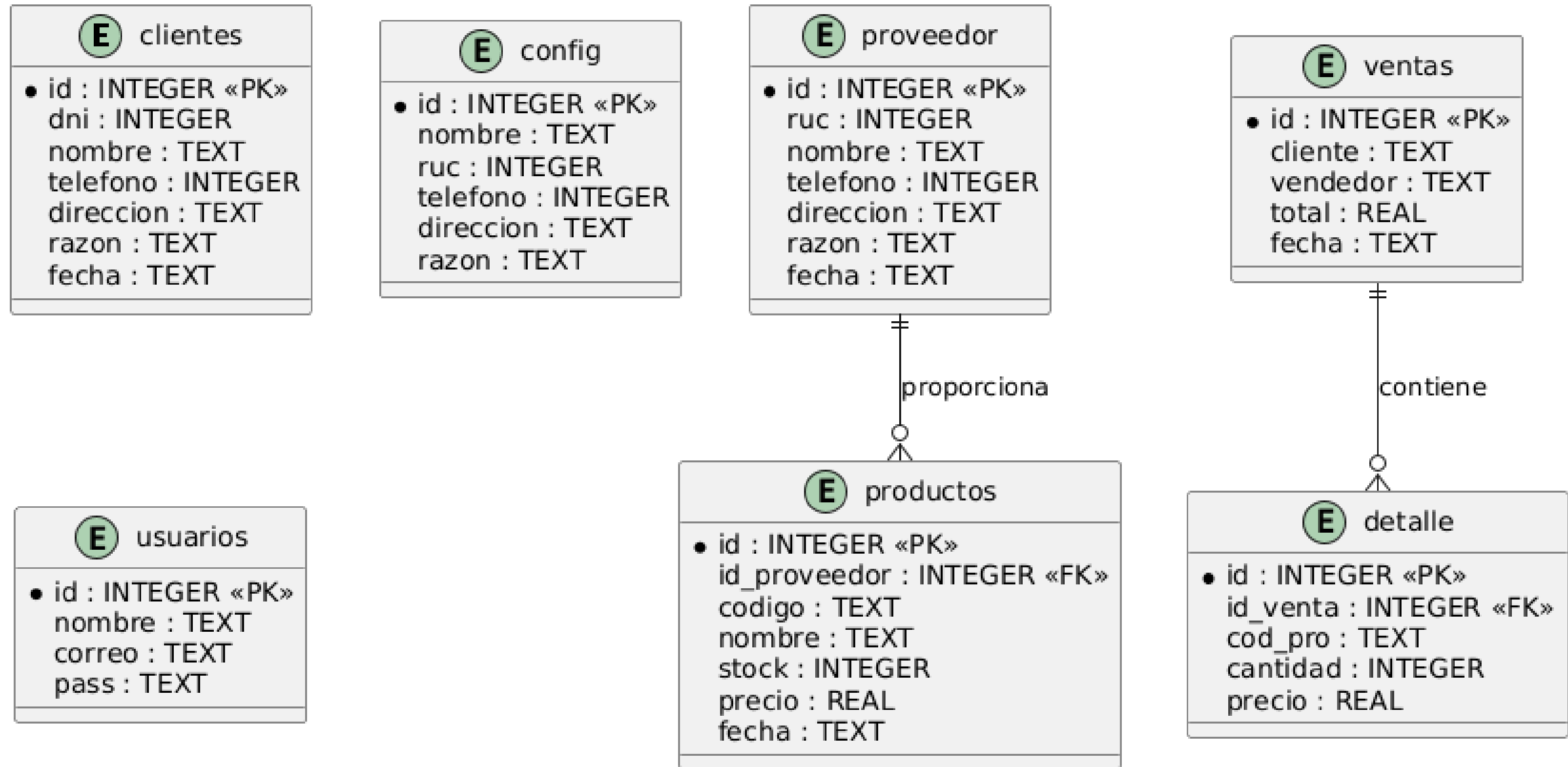
public class Conexion {
    Connection con;

    public Connection getConnection() {
        try {
            String url = "jdbc:sqlite:src/database/sistemaventa_COMPLETA.db";
            con = DriverManager.getConnection(url);

            Statement stmt = con.createStatement();
            stmt.execute("PRAGMA foreign_keys = ON");
            stmt.close();

        } catch (SQLException e) {
            System.out.println("Error de conexión: " + e.getMessage());
        }
        return con;
    }
}
```


DIAGRAMA ENTIDAD-RELACION



ENTIDADES

E clientes

- id : INTEGER «PK»
dni : INTEGER
nombre : TEXT
telefono : INTEGER
direccion : TEXT
razon : TEXT
fecha : TEXT

E config

- id : INTEGER «PK»
nombre : TEXT
ruc : INTEGER
telefono : INTEGER
direccion : TEXT
razon : TEXT

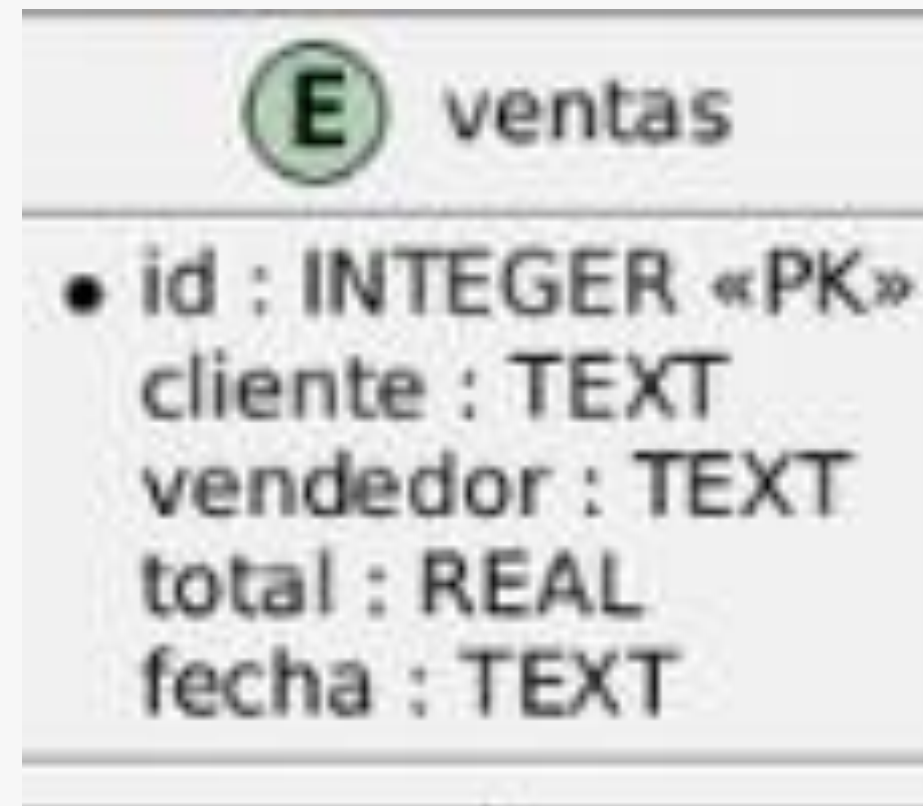
E proveedor

- id : INTEGER «PK»
ruc : INTEGER
nombre : TEXT
telefono : INTEGER
direccion : TEXT
razon : TEXT
fecha : TEXT

E usuarios

- id : INTEGER «PK»
nombre : TEXT
correo : TEXT
pass : TEXT

ENTIDADES





Cliente

INSERT

```
public boolean RegistrarCliente(Cliente cl) {  
    String sql = "INSERT INTO clientes (dni, nombre, telefono, direccion, razon) VALUES (?, ?, ?, ?, ?)";
```

READ

```
public List<Cliente> ListarCliente() {  
    List<Cliente> ListaCl = new ArrayList<>();  
    String sql = "SELECT * FROM clientes";
```

UPDATE

```
public boolean ModificarCliente(Cliente cl) {  
    String sql = "UPDATE clientes SET dni = ?, nombre = ?, telefono = ?, direccion = ?, razon = ? WHERE id = ?";
```

DELETE

```
public boolean EliminarCliente(int id) {  
    String sql = "DELETE FROM clientes WHERE id = ?";
```



Productos

INSERT

```
public boolean RegistrarProductos(Productos pro) {  
    String sql = "INSERT INTO productos (codigo, nombre, proveedor, stock, precio) VALUES (?, ?, ?, ?, ?)";
```

READ

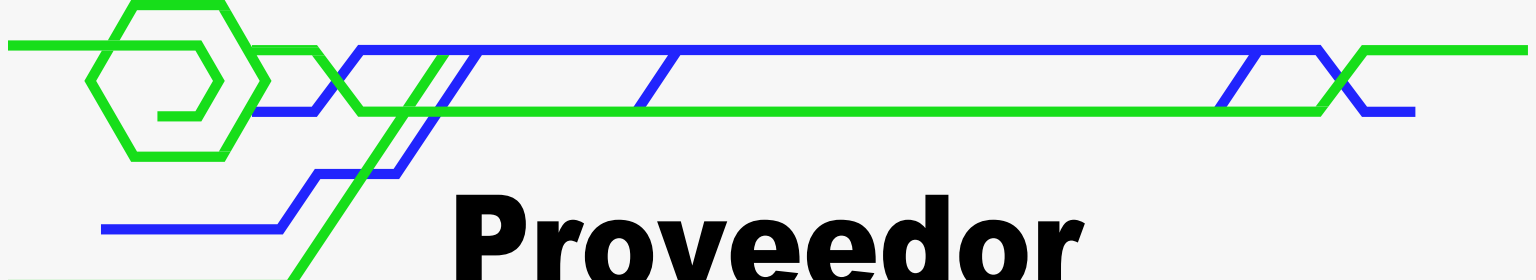
```
public void ConsultarProveedor(JComboBox proveedor) {  
    String sql = "SELECT nombre FROM proveedor";
```

UPDATE

```
public boolean ModificarProductos(Productos pro) {  
    String sql = "UPDATE productos SET codigo = ?, nombre = ?, proveedor = ?, stock = ?, precio = ? WHERE id = ?";
```

DELETE

```
public boolean EliminarProductos(int id) {  
    String sql = "DELETE FROM productos WHERE id = ?";
```



Proveedor

INSERT

```
public boolean RegistrarProveedor(Proveedor pr) {  
    String sql = "INSERT INTO proveedor(ruc, nombre, telefono, direccion, razon) VALUES (?, ?, ?, ?, ?)";
```

READ

```
public List<Proveedor> ListarProveedor() {  
    List<Proveedor> Listapr = new ArrayList<>();  
    String sql = "SELECT * FROM proveedor";
```

UPDATE

```
public boolean ModificarProveedor(Proveedor pr) {  
    String sql = "UPDATE proveedor SET ruc = ?, nombre = ?, telefono = ?, direccion = ?, razon = ? WHERE id = ?";
```

DELETE

```
public boolean EliminarProveedor(int id) {  
    String sql = "DELETE FROM proveedor WHERE id = ?";
```



Ventas

INSERT

```
// Registra una nueva venta en la tabla 'ventas'  
public int RegistrarVenta(Venta v) {  
    String sql = "INSERT INTO ventas (cliente, vendedor, total) VALUES (?, ?, ?)";
```

SELECT

```
// Devuelve una lista de todas las ventas realizadas  
public List<Venta> Listarventas() {  
    List<Venta> ListaVenta = new ArrayList<>(); // Lista vacía para almacenar ventas  
    String sql = "SELECT * FROM ventas"; // Consulta todas las ventas
```

UPDATE

```
// Actualiza el stock de un producto después de realizar una venta  
public boolean ActualizarStock(int cant, String cod) {  
    String sql = "UPDATE productos SET stock = ? WHERE codigo = ?";
```

GRACIAS POR SU ATENCION !

